

Kubernetes HorizontalPodAutoscaler Not Scaling Runbook

Document ID: RB-K8S-009 | Version: 1.0 | Last Updated: 2024-11-15

Category	kubernetes	Severity	P2 - High
Domain	Container Orchestration	Est. Duration	20-35 minutes
Tags	kubernetes, hpa, autoscaler, metrics-server, cpu, scaling		

Overview

A HorizontalPodAutoscaler that is not scaling may leave the application under-provisioned during traffic spikes. Common causes: metrics-server not running, missing resource requests on pods, wrong metric type, or cooldown/stabilization window preventing scale-out.

Prerequisites

- kubectl configured for the cluster
- Access to metrics-server pod logs
- Understanding of the expected traffic pattern

Response Steps

Step 1: Check HPA Status

Describe the HPA to see current metrics values, desired replicas, and any error conditions.

```
kubectl get hpa -n
kubectl describe hpa -n
```

NOTE: Look for 'unknown' in the TARGETS column — this means HPA cannot read metrics.

Step 2: Verify Metrics Server is Running

The HPA requires metrics-server to be running in kube-system to read CPU/memory metrics.

```
kubectl get pods -n kube-system | grep metrics-server
kubectl top pods -n
kubectl top nodes
```

NOTE: If kubectl top fails, metrics-server is not running or has certificate issues.

Step 3: Check Resource Requests are Set

HPA CPU percentage targets are calculated relative to the pod's CPU request. If requests are not set, HPA cannot calculate utilization.

```
kubectl get deployment -n -o jsonpath='{.spec.template.spec.containers[*].resources}'
kubectl describe hpa -n | grep -i 'resource'
```

NOTE: CRITICAL: Set CPU requests on all containers. Without requests, HPA shows unknown/0% utilization.

Step 4: Check HPA Min/Max Replicas

Verify the HPA min and max replica bounds are not preventing scaling.

```
kubectl get hpa -n -o jsonpath='{.spec}'
kubectl get deployment -n -o jsonpath='{.spec.replicas}'
```

Step 5: Check Stabilization Window

The default HPA stabilization window is 5 minutes for scale-down, 0 for scale-up. Check if scale-up is blocked by the window.

```
kubectl get hpa -n -o yaml | grep -A10 'behavior'
kubectl describe hpa -n | grep -A5 'ScaleUp'
```

Step 6: Trigger Manual Scale and Patch HPA

If HPA is stuck, manually scale the deployment and patch the HPA to trigger a re-evaluation.

```
kubectl scale deployment --replicas= -n
kubectl patch hpa -n -p '{"spec":{"minReplicas":}}'
kubectl get hpa -n -w
```

Rollback Steps

Rollback Step 1: Disable HPA and Use Manual Replicas

If HPA is causing instability, delete it and revert to manual replica management.

```
kubectl delete hpa -n
kubectl scale deployment --replicas= -n
```

Rollback Step 2: Reinstall Metrics Server

If metrics-server is broken, reinstall it via the official manifest.

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl rollout status deployment/metrics-server -n kube-system
```

Step Dependencies

Step	Depends On	Can Parallelize With
Step 1: Check HPA Status	—	—
Step 2: Verify Metrics Server	Step 1	Step 3
Step 3: Check Resource Requests	Step 1	Step 2
Step 4: Check Min/Max Replicas	Step 1	Step 2, Step 3
Step 5: Check Stabilization Window	Step 1	Step 2, Step 3, Step 4
Step 6: Manual Scale + Patch	Step 2, Step 3, Step 4, Step 5	—

Maintained by Platform Engineering. For updates ping #platform-oncall.